



MX1 CXL Computational Memory Device

A Type 3 CXL device that unifies memory expansion, SSD-backed memory, and near-memory processing

Harry Kim XCENA Inc.

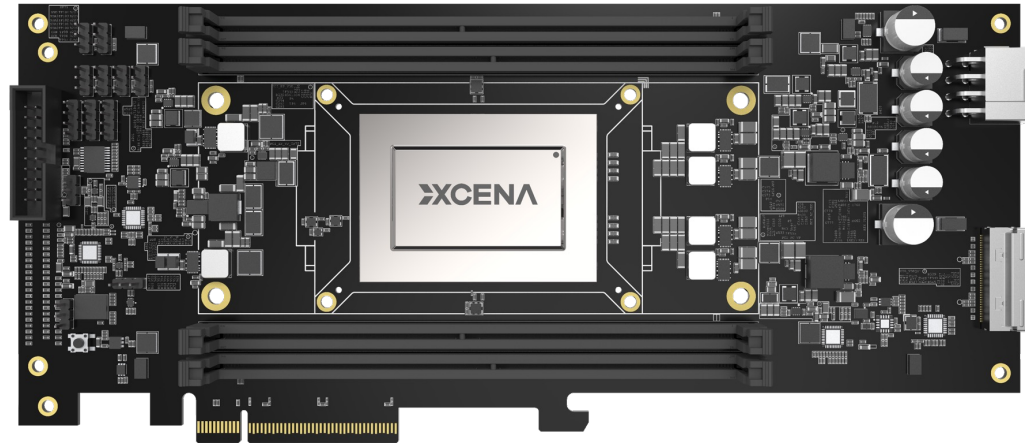
Jinin So Samsung Electronics

Harry Kim · Dohun Kim · Sungwoo Chang · Grant Mackey · Brian Hirano · Nam Sung Kim (XCENA) |
Jinin So · Wonjae Lee · Seunghak Lee · Hojin Nam · Jinoh Ahn (Samsung Electronics)

(Memory Xcelerator)

MX1: Expand, Extend, Compute

Memory is intelligence. AI workloads are bound by memory capacity, bandwidth, and power efficiency rather than raw compute.



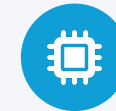
CXL Memory Expansion

- 4ch DDR5 8400MT/s, up to 2TB
- CXL 3.2 Type 3, PCIe Gen6
- Chip-kill RAS



Infinite Memory Extension

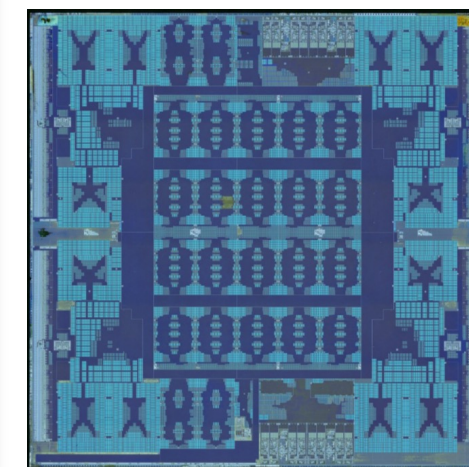
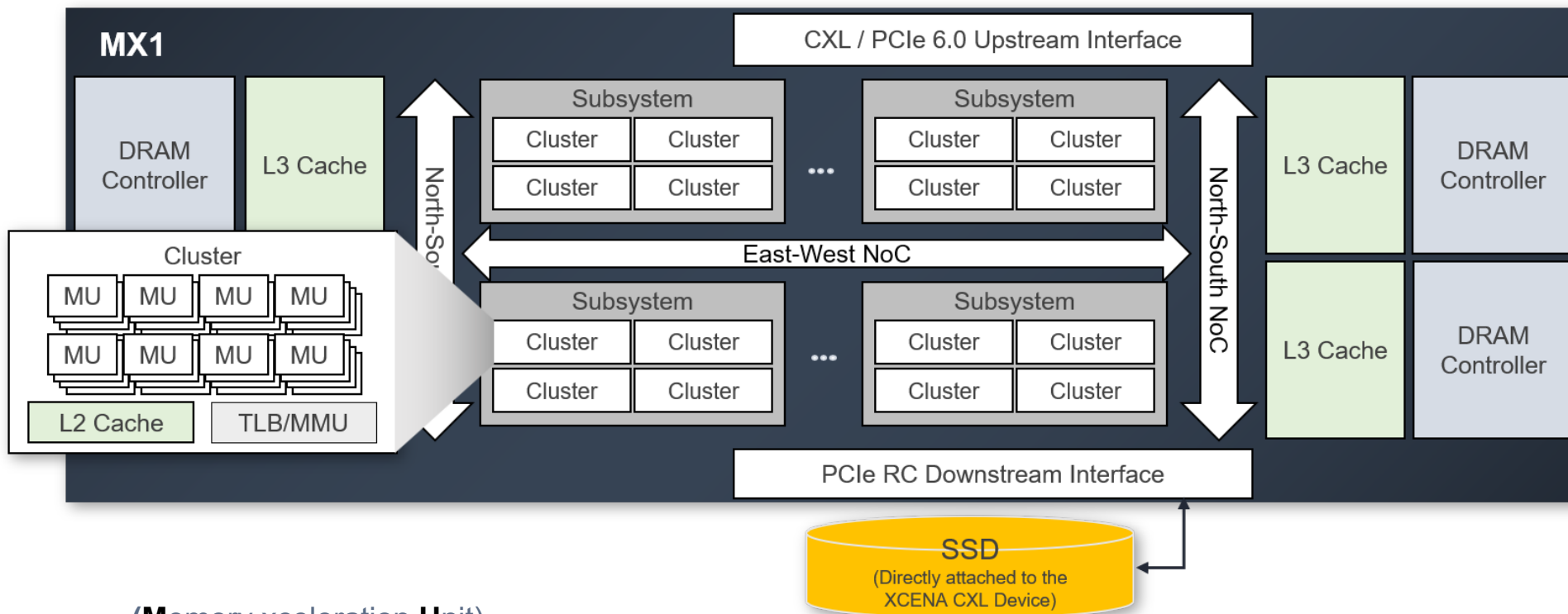
- SSD on MX1 root port = CXL memory
- Host sees full SSD capacity
- DRAM as cache tier



Near-Memory Compute

- 3,072 custom RISC-V cores
- User-defined kernels
- SDK available — xcena.com/SDK

MX1 system architecture



Samsung Foundry 4nm

(Memory xceleration Unit)

32 MUs per Cluster

- 64-bit RISC-V for integer, FP32/64
- Shared L2 cache
- V2P translation

x4 Clusters per Subsystem

- Shares resources and scheduling
- Subsystem is the unit of host job allocation

x24 Subsystem = 3072

- Runs 24 independent jobs
- 128MB shared L3 cache
- Connected by in-house NoC

Many simple cores and RISC-V



Target workloads: Data parallel executions

In-memory data analytics, RAG retrieval, memory compression, and more
Aggregate memory-bandwidth and energy efficiency matter more than single-thread speed.



Many simple cores

- Chosen over fewer BIG cores
- Built for bandwidth-bound, parallel kernels
- More aggregate performance per watt



PPA efficiency

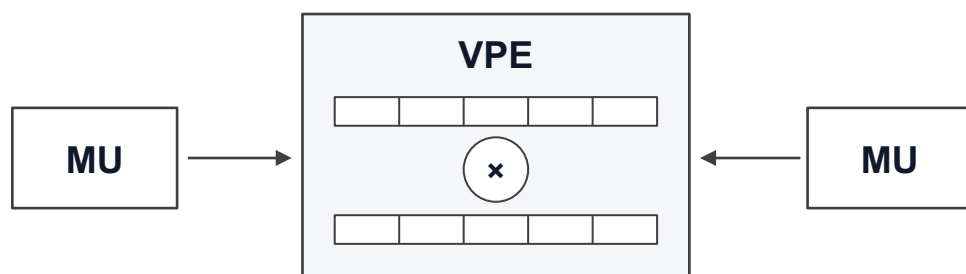
- In-order issue cuts area and power
- Fits 3072 cores on the die
- Keeps energy-per-byte low



RISC-V open ISA

- Open ISA, mature toolchain
- Established software ecosystem
- Extended with custom instructions

Vector Processing Engine: Vector accelerator



- Custom MU ISA extensions, HW DMA engines, dedicated SRAM buffers
- Shared across 4 clusters; concurrent multi-MU-group use
- Per-MU command queues → fine-grained, parallel scheduling
- ~3 TFLOPS peak per MX1 SoC (FP32/FP16 dot-product)

Supported vector operations

Element-wise arithmetic

Reduction operations such as dot product

Vector-scalar arithmetic & bitwise XOR

Equality and all-zero checks on vectors

CRC32 computation over arrays

Applications

🔍 RAG vector search

🗄️ KV cache scoring & offload

🔽 Feature preprocessing

MX1 memory architecture

I-cache

L1 I\$ 8 KB shared by 4 MUs

Sized for a kernel's inner-loop footprint

L2 I\$ 128 KB per cluster

Shared by all MUs in the cluster

Instructions use physical addresses.

D-cache

L1 D\$ 4 KB, 4-way, Virtual

Dedicated per MU

L2 D\$ 256 KB, 32-way, VIPT

Per **cluster**, shared by 32 MUs

*Data uses **HOST** virtual addresses.*

D-TLB @ L2

TLB 1024 entries · 64 KB pages

Shared across all 32 MUs in the cluster

+ 8 entries for 1 GB pages

Huge pages cover accesses over wide regions

V2P translation is shared at the cluster.

Shared LLC 128 MB, 32-way, unified (data + instructions), shared by all clusters.



- Same API as the standard malloc-free library
- Existing code moves to CXL memory with minimal change

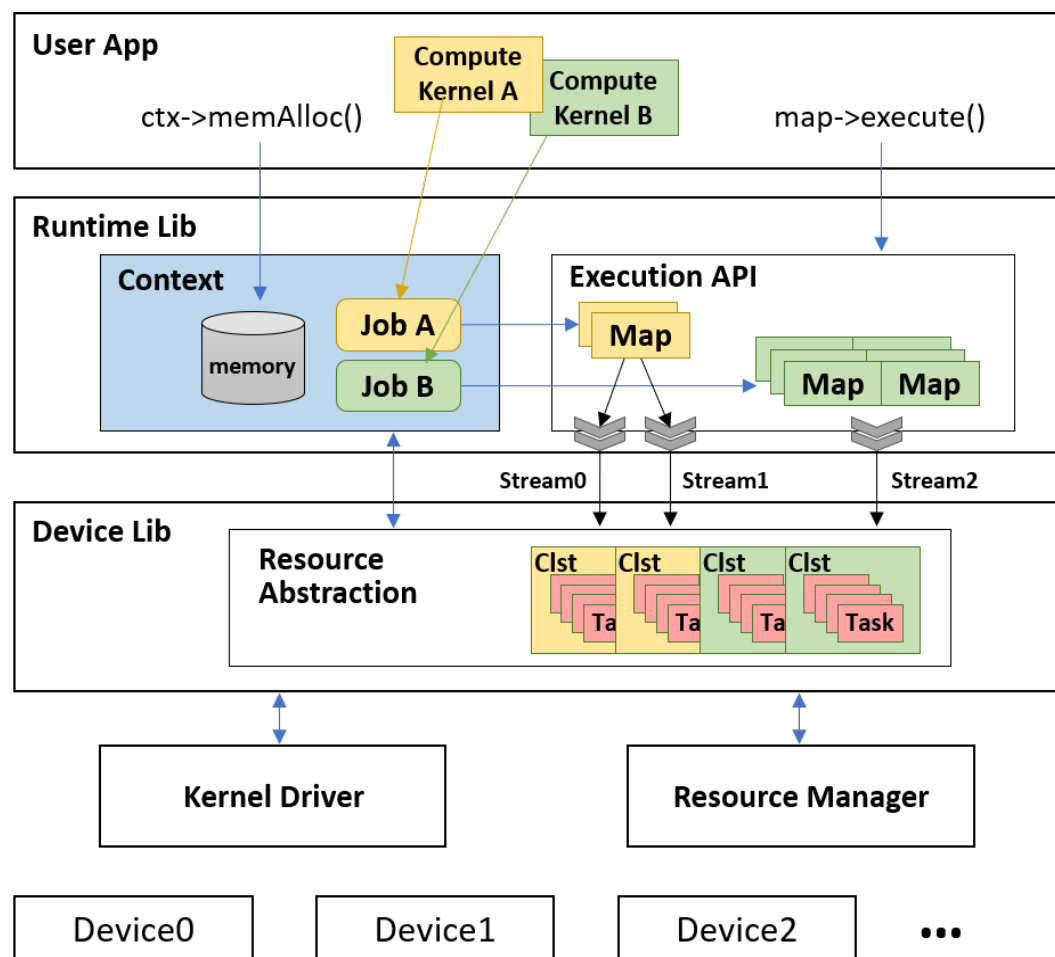


- Page table maps Host VA to PA
- Host and MU see identical addresses
- Pointer-rich structures traverse safely on both sides



- Each app gets its own page table
- One app's kernels cannot touch another's CXL memory

PXL: a MapReduce-style runtime for the MUs



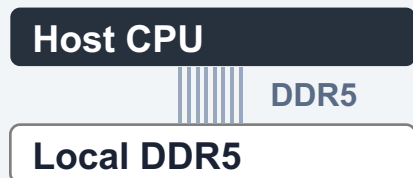
The **Parallel Xceleration Library** exposes the interfaces that applications use to allocate resources and run kernels on the MUs.

- 1 Device context**
 Recognizes the MX1 device
 Abstracts memory/compute into a control interface
- 2 Compute Kernels**
 Write MU kernels in C/C++ or Rust
 Follow the data-parallel execution model
- 3 Jobs**
 Job: parallel resources running one kernel
 Allocates one or more MU Subsystems
- 4 Map / tasks**
 Map API splits work into tasks across MU cores
 PXL runtime handles allocation and sync automatically

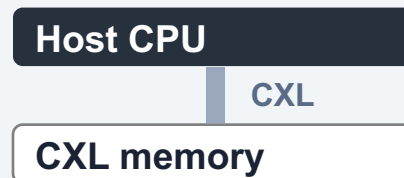
More throughput at lower power – analytics kernels

■ = where computation runs

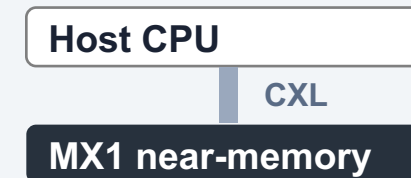
Host Local DRAM



Host over CXL



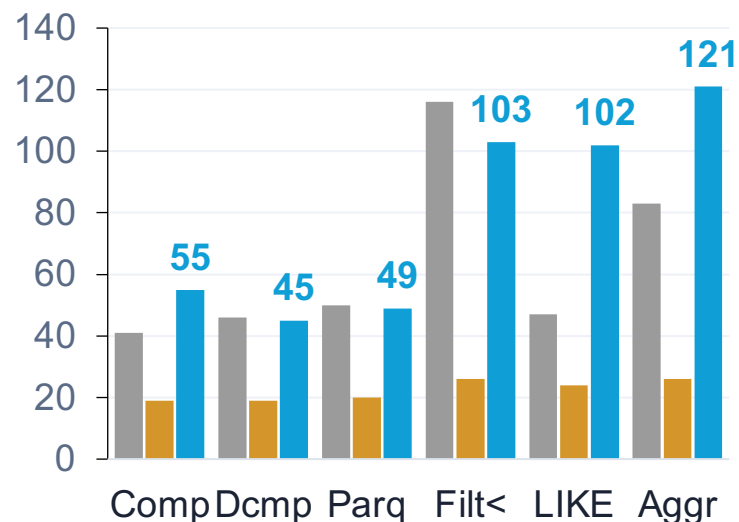
MX1 Offload



Throughput (GB/s)

up to 4.7× vs CXL, 2× vs DRAM

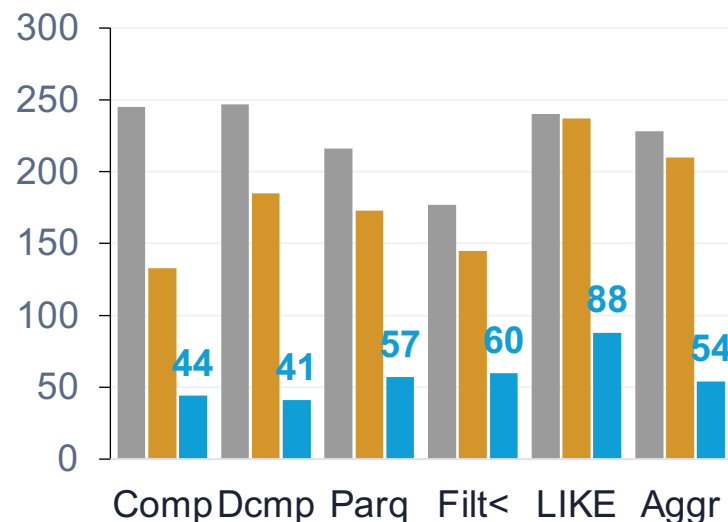
■ Local DRAM ■ over CXL ■ MX1



Dynamic Power over Idle (W)

about a quarter of host power

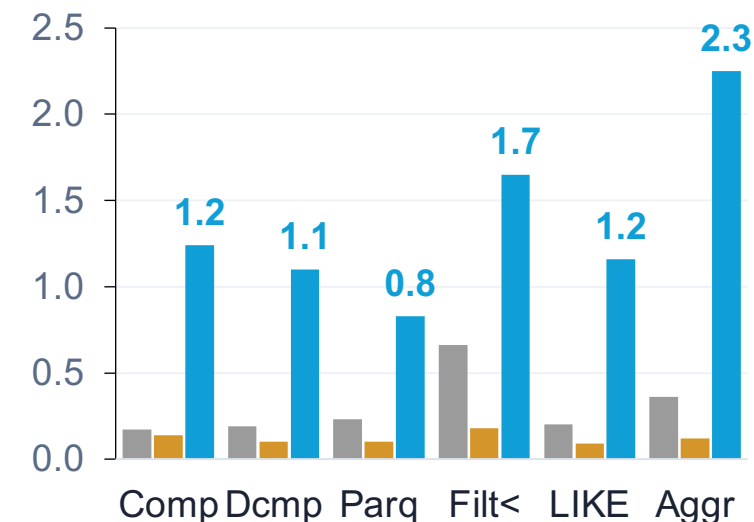
■ Local DRAM ■ over CXL ■ MX1



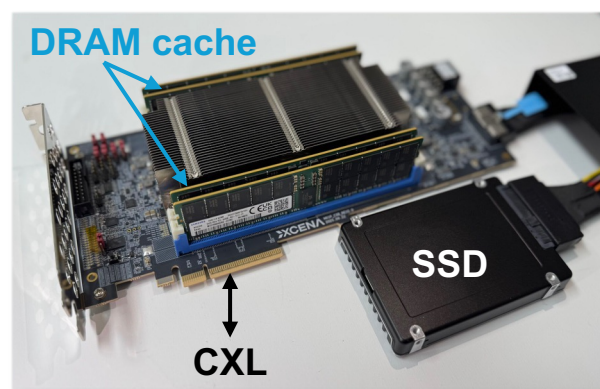
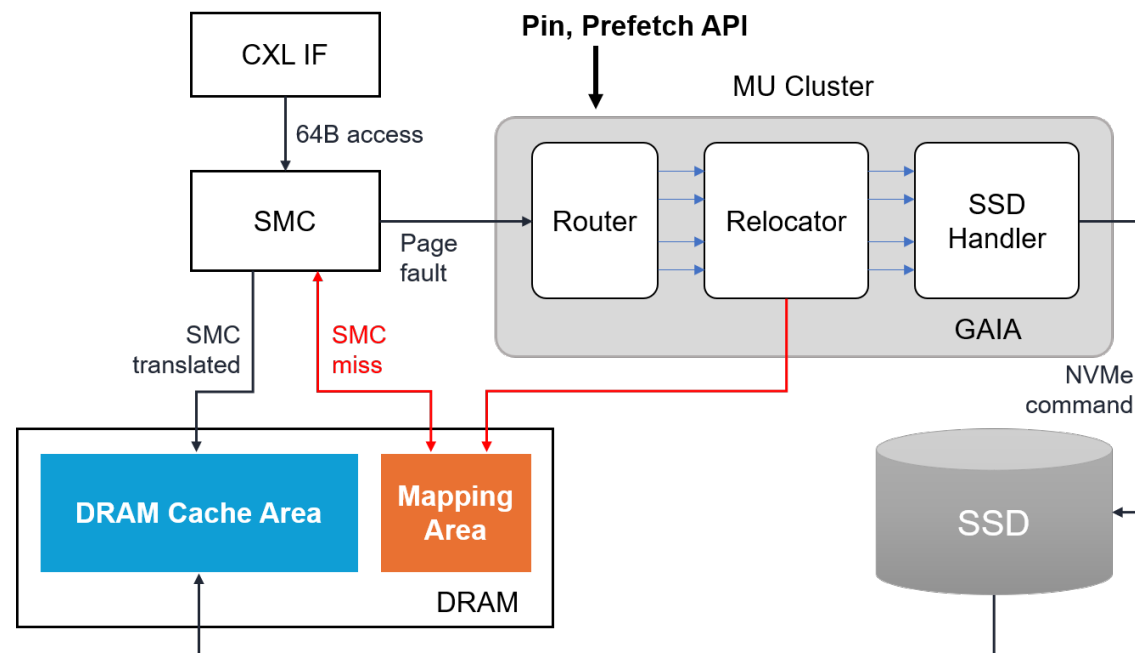
Efficiency (GB/J)

up to 18.7× vs CXL, 6.2× vs DRAM

■ Local DRAM ■ over CXL ■ MX1



Presenting **SSD capacity** as byte-addressable CXL memory



Infinite Memory (CMM-H device)

Uses DRAM+NAND hybrid memory media
Infinite Memory spans full SSD capacity



DRAM caches SSD pages

Reads/writes served from DRAM
SSD split into 64 KB pages with mapping entries



SSD Map Cache hides mapping lookups

1024-entry Map Cache (TLB) locates DRAM pages
Misses filled by firmware on MU cores

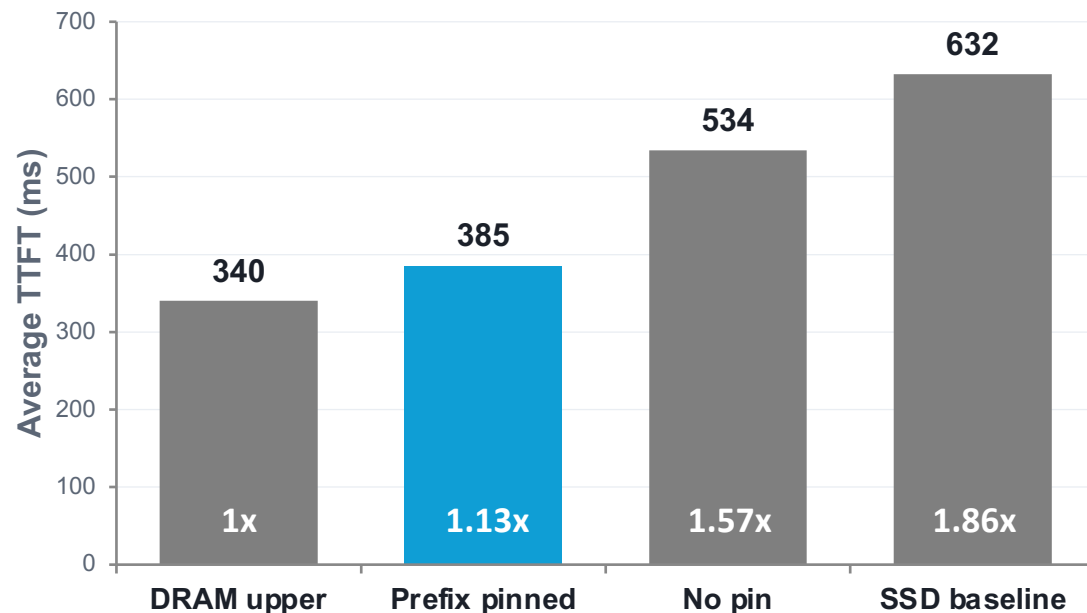


64 KB page granularity

Balances mapping overhead and performance
Aligns TLC NAND structure

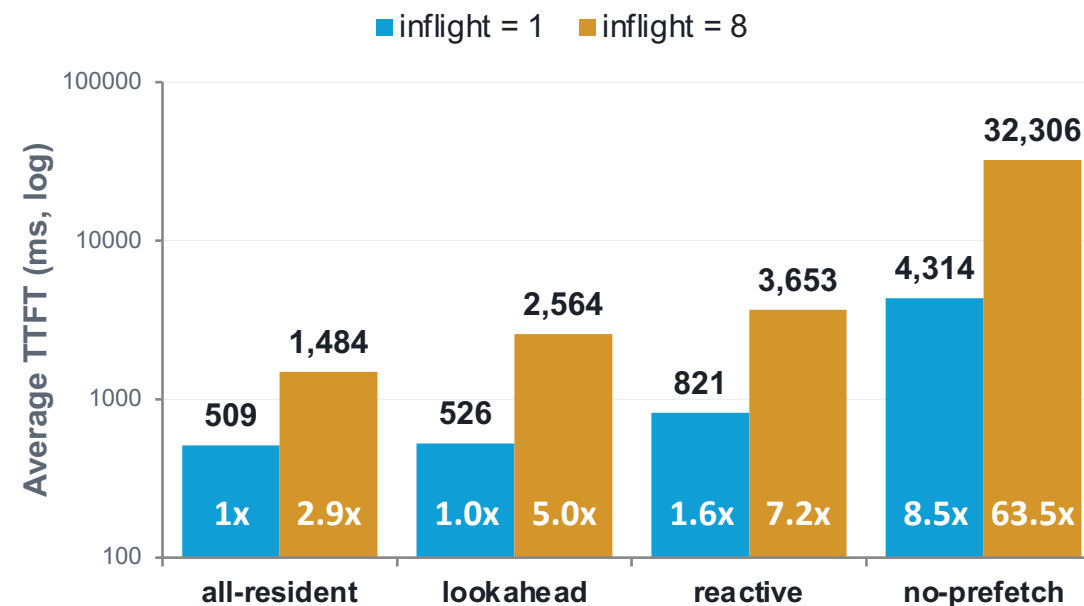
Infinite Memory applied to KV cache

Pinned prefix $\approx$ DRAM Queries against the shared document



- Pinned prefix stays resident, so query TTFT tracks DRAM (1.13x).
- Unpinned drops to 1.57x; raw SSD to 1.86x. Repeated prefix reads are eliminated.

Lookahead prefetch Warm sessions, inflight 1 and 8



- Lookahead prefetch exploits device under-utilization to deliver DRAM-resident performance.
- With the right DRAM capacity and prefetch policy, storage overhead is effectively eliminated.

Up next: Samsung

CXL-PNM use cases for AI inference

-  Realizing rack-scale CXL systems
-  RAG vector search
-  GPU-PNM hybrid attention

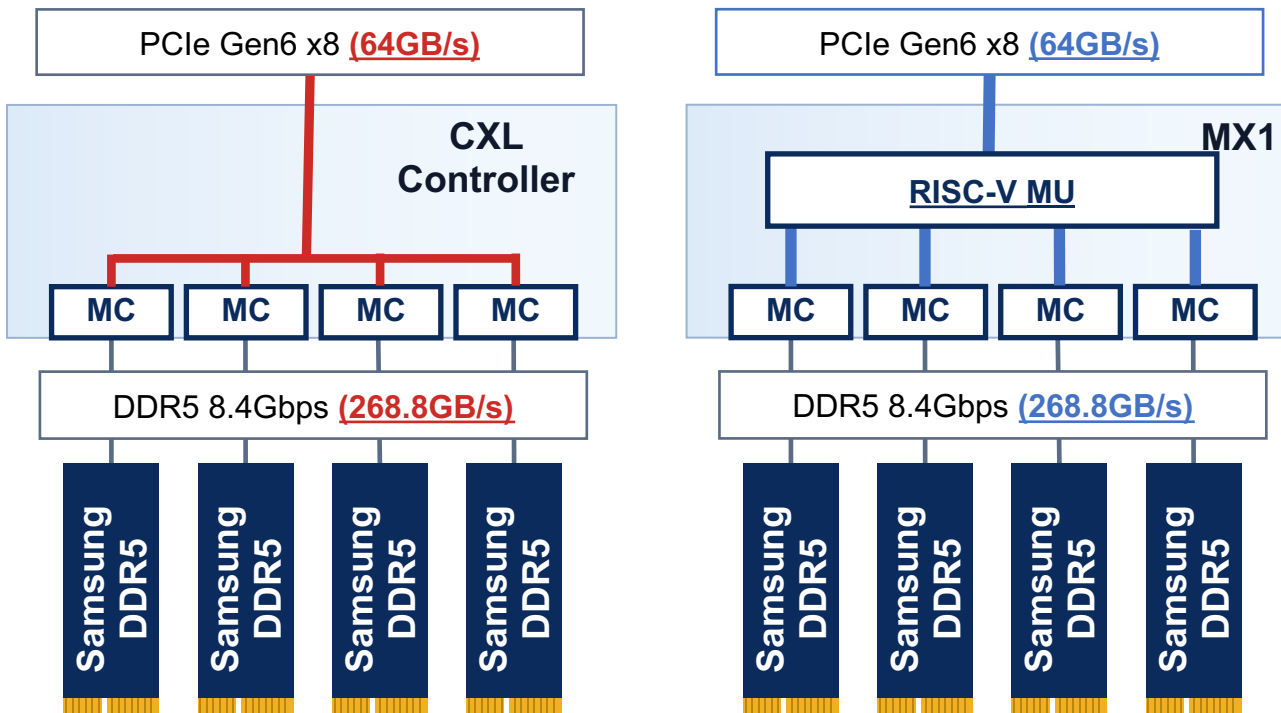
XCENA × Samsung joint session

(Processing-Near-Memory)

Samsung CXL-PNM Research with XCENA MX1

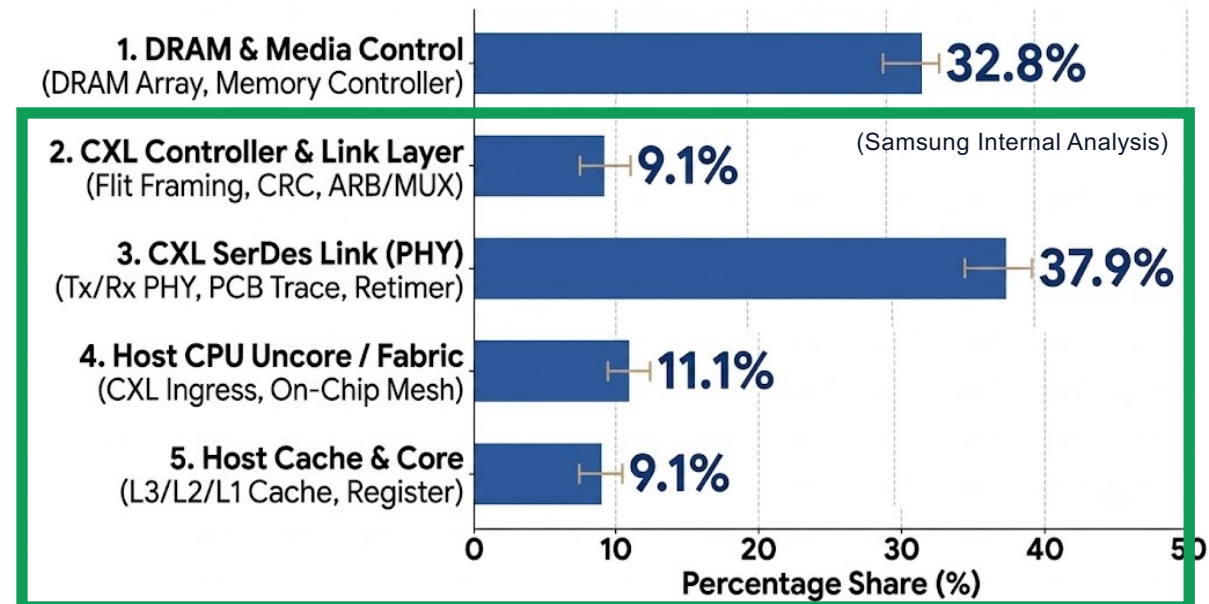
- Maximize Utilization of **huge DRAM Bandwidth** behind CXL Interconnect.
- Compute In-place and **ONLY Reduced Data Transferred**. → Data transfer **energy reduction**.

Maximize DRAM BW behind CXL



Minimize Data Transfer Energy

CXL Memory Expander Energy Breakdown (%)



Data transfer energy(pJ/bit) between CXL Controller to Host (67.2%) can be minimized with PNM.

Samsung-XCENA CXL Computational Memory System

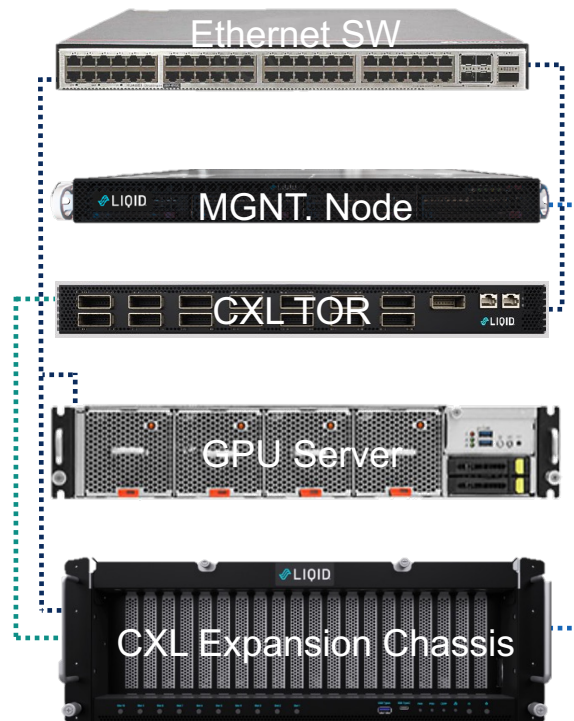
- GPU Servers can share this memory system through CXL memory semantic directly.
- LLM KV cache offloading(pooling, sharing) with memory semantic, RAG Vector DB
- Memory Capacity : **20TB**, Memory Bandwidth : **2.7TB/s**

Rack-Scale CXL Appliance

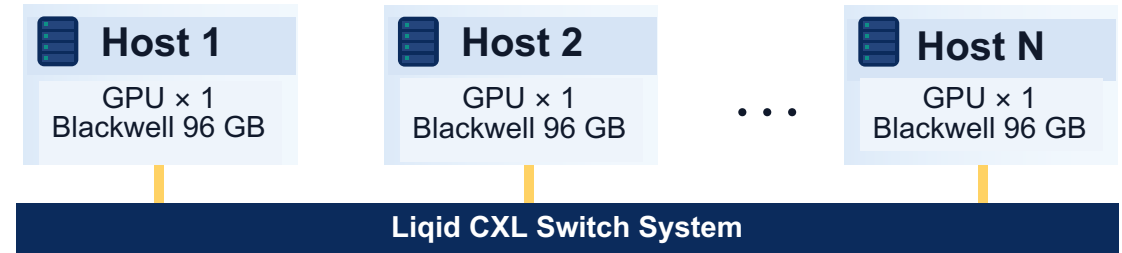


XCENA MX1 - Hot Chips 2026

Building Blocks



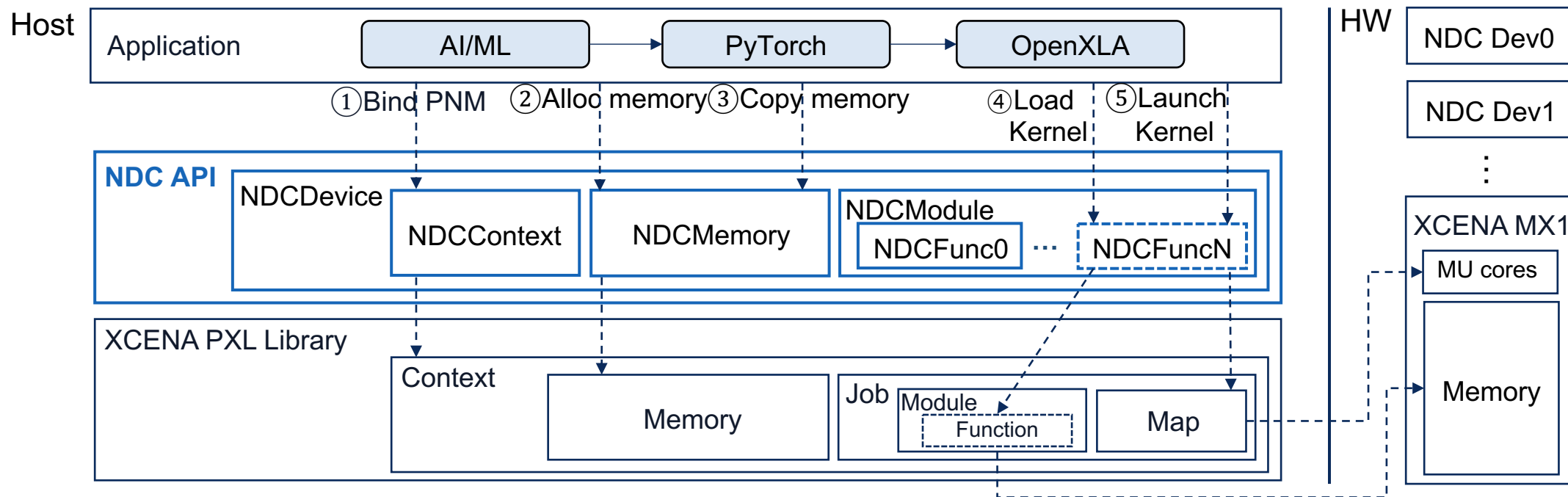
System Topology



Samsung-XCENA SW Runtime Approach for PNM: NDC API

(Near-Data-Computing)

- NDC API¹⁾: Samsung vendor-agnostic library for programming PNM devices
- **Open-sourced** via OCP FTI DCC, released to accelerate Near-Data-Computing ecosystem.
- PyTorch integration in progress via OpenXLA and PrivateUse1 with **no code change**.
(PyTorch extension for new device)



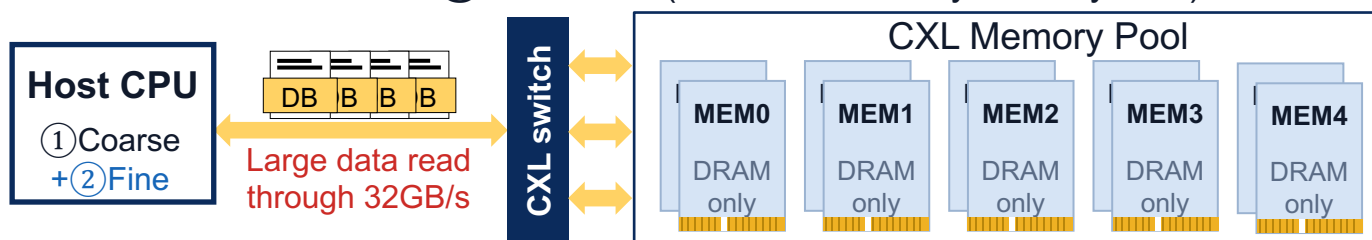
AI Use Case 1 - RAG Vector Search on CXL-PNM

- IVF Flat indexing using FAISS Library, offloading L2-kNN computations to CXL PNM devices
- L2-kNN distance computation is memory-bandwidth-bound

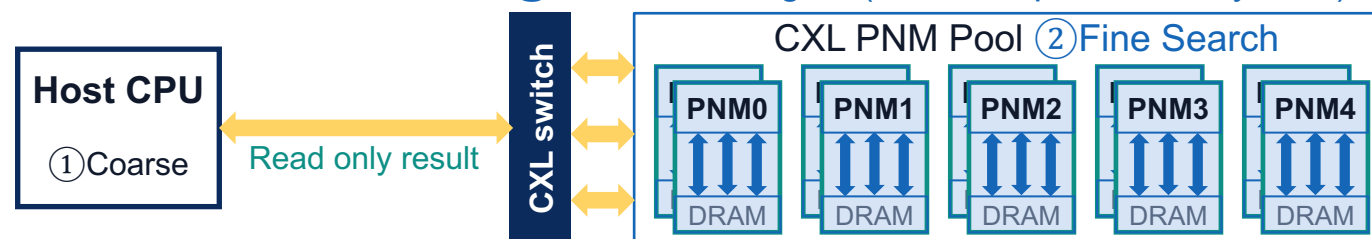
FAISS Index Search Workflow

Switch Interconnect(PCIe Gen5 x8, 32GB/s) On-Device Interconnect per dev(200GB/s)

Baseline - Vector Search @ Host CPU (with CXL Memory Pool System)



PNM Offload - Vector Search @ CXL PNM Engine (CXL Compute Memory Pool)

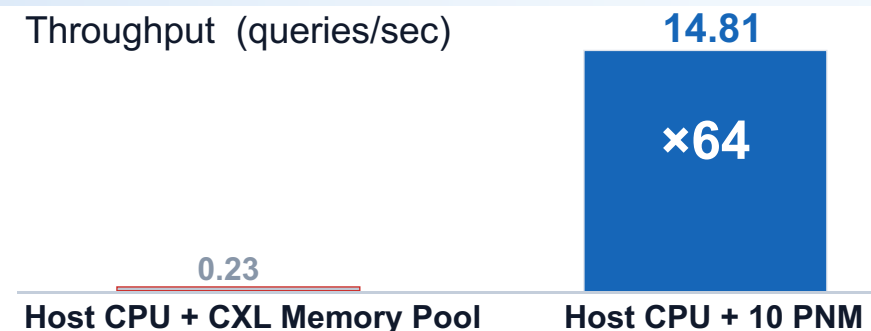


Index: IVF-Flat (FAISS), 512M total vectors, nlist=6144, nprobe=1920

Dataset: Laion5B, convert to 1024-dimension vector using CLIP-ViT-H-14, sample to 4.5TB vectors

XCENA MX1 - Hot Chips 2026

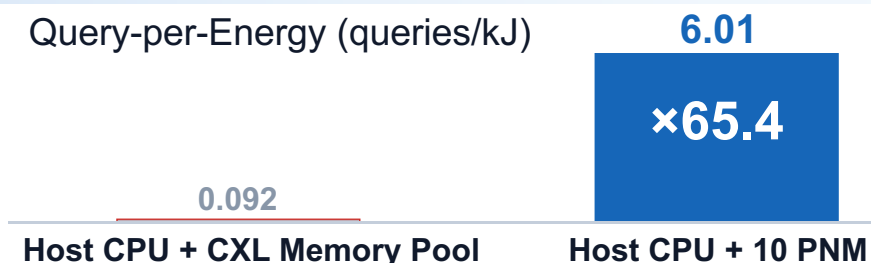
10 PNM devices deliver 64× higher QPS



Effective Read BW utilization

- Host CPU Read BW: 26 GB/s
- 10x PNM aggregate BW: 1625GB/s

65× better query-per-energy



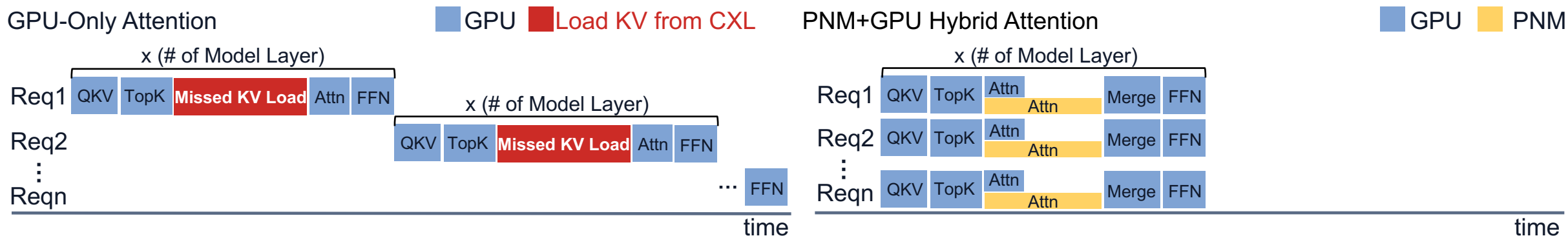
AI Use Case 2 - LLM Decode: GPU + PNM Hybrid Attention

- Normally, KV cache Transfer time > calculation time on GPU
- In decode stage, instead of loading all KV pages, delivering only attention result calculated by PNM

Hybrid Attention - Hit Pages on GPU, Miss Pages on PNM in decode attention



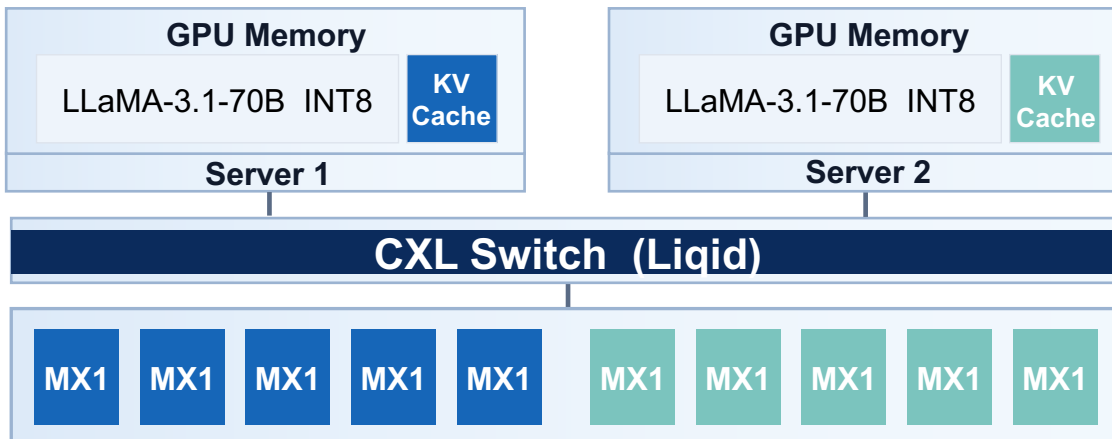
Decode Timeline - GPU-Only Attention vs Hybrid Attention (batch size=n)



AI Use Case 2 - LLM Decode: GPU + PNM Hybrid Attention

- KV cache Data transfer time is **much longer** than PNM compute time @ multi-batch workload.
- At 100K context, using the PNM pool improves throughput by **3.35x** and energy efficiency by **3.84x**.

Experimental Setup (2 servers, 5 PNM / server)



Model : LLaMA 3.1 70B, DP=2, with INT8 quantized weights(70GB)

Metrics : 100K context decode step latency

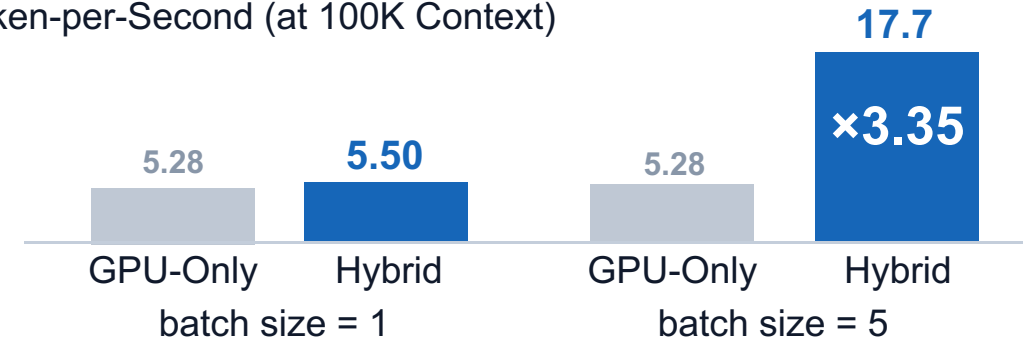
Context : 100K tokens per request

GPU : NVIDIA RTX PRO 6000 Blackwell (96 GB)

GPU KV Cache Size : 8GB per GPU

Hybrid attention gives 3.35x throughput at 100K context

Token-per-Second (at 100K Context)



3.84x better token-per-energy

Token-per-Energy (Toks/kJ)



CXL-PNM delivers real gains on real silicon

- ✓ MX1 is a CXL 3.2 computational memory device with many-core RISC-V and unified virtual addressing
- ✓ XCENA complements MX1 with PXL for PNM programming and Infinite Memory for cost-effective expansion
- ✓ Samsung realized a rack-scale CXL computational memory system in collaboration with XCENA
- ✓ RAG vector search & GPU+PNM hybrid attention deliver real inference gains

Thank you

XCENA

contact@xcena.com / xcena.com/sdk

Samsung

jinin.so@samsung.com

XCENA × Samsung joint session